Infon: A Knowledge Graph Reasoner with Calibrated Uncertainty

Eden Duthie
Trusst.ai & AgenticX
eduthie@trusst.ai
eduthie@agenticx.org

Charles Prosper
Amazon Web Services

Josh Passenger
Amazon Web Services

May 2026

Abstract

Infon is a knowledge graph reasoner that performs fact-checking over document corpora with calibrated uncertainty. Three properties distinguish it from prior systems: *cassette-native storage* (immutable, content-addressed, S3-native documents with manifest pruning), *honest uncertainty* via a first-class Dempster–Shafer vacuous element $m(\Theta)$ that rises to 1.0 on unsupported claims rather than fabricating confident answers, and *MCTS multi-hop reasoning* guided by a frozen sheaf graph neural network as a terminal scorer. On the actor-to-actor real-data evaluation, the symbolic pipeline scores 80% accuracy; adding the sheaf GNN raises this to 100%. On synthetic held-out data the GNN reaches 99.2% overall accuracy. The system operates on a laptop CPU without GPU or API keys. Code is released under the repository tag `preprint`.

1 Introduction

Fact-checking over document corpora is a central task in applied NLP: given a claim and a collection of source documents, a system must decide whether the corpus supports the claim, refutes it, or provides no relevant evidence. Current approaches fall into two failure modes. Retrieval-augmented generation (RAG) systems retrieve supporting passages and feed them to a language model, which can *hallucinate* verdicts on claims the corpus does not speak to Lewis et al. [2020]. Knowledge-graph pipelines that extract structured triples achieve better grounding but require rigid, hand-curated schemas and cannot express that a question simply lacks an answer in the available evidence Hogan et al. [2021].

Neither class of system handles *multi-hop* reasoning with *calibrated uncertainty*: the ability to chain intermediate facts across documents while simultaneously signalling when the chain is absent or incomplete. This gap is consequential. A compliance analyst who asks whether company A indirectly supplies component B via an intermediate supplier needs both the chain and an honest "the corpus does not say" when the chain is missing.

This paper. We present **Infon**, a deployable knowledge graph reasoner that fills this gap. Infon is not a neural language model; it is a symbolic reasoner augmented by a learned prior. Its design is guided by five engineering properties:

1. **Cassette-native storage.** Documents are stored as immutable, content-addressed `.inf` files that can live on S3 or a local filesystem via `fsspec`. Delta ingests append; nothing is rewritten. A manifest pruner skips cassettes that cannot contain the query anchor, cutting Parquet opens by 7–16× at scale and 278× on cache misses.

2. **Schema-free operation.** Plain-English questions are encoded with SPLADE-tiny Formal et al. [2021] (17 MB, Apache 2.0, no GPU) to an anchor vocabulary. Connective predicates are auto-inferred from the data: no ontology annotation is required. A Strands-powered `Analyst` agent routes natural-language questions to the appropriate retrieval primitive, citing every source it uses.

3. **Dempster–Shafer mass with first-class $m(\Theta)$.** Every verdict is a four-part mass $(m_S, m_R, m_U, m_\Theta)$ over the frame $\Omega = \{S, R, U\}$ (Supports, Refutes, Uncertain), where $m_\Theta = m(\Omega)$ is the vacuous element that rises to 1.0 on unsupported claims rather than a confident wrong answer Shafer [1976].

4. **MCTS and sheaf GNN multi-hop reasoning.** A Monte Carlo Tree Search traverses the infon hypergraph, with a 140k-parameter frozen sheaf graph neural network Bodnar et al. [2022] serving as the terminal scorer. The GNN provides the MCTS prior that guides search toward high-confidence chains.

5. **Schema evolution without reingestion.** A categorical `SchemaFunctor` rewrites existing cassettes under a new ontology via a Kan pushforward. Old snapshots remain queryable; the migration is 62× faster than re-extraction.

Roadmap. Section 2 establishes the formal background: Dempster–Shafer theory, situation semantics and infons, SPLADE sparse retrieval, and sheaf graph neural networks. Section 3 describes Infon's four-layer architecture (cassette substrate, query DSL, reasoner, and conversational analyst). Section 4 details the implementation, including the synthgen corpus, GNN training, and manifest pruner benchmarks. Section 5 reports experimental results on synthetic and real-world evaluations (HoVer Jiang et al. [2020], AVeriTeC Schlichtkrull et al. [2023], SciFact Wadden et al. [2020]). Section 6 discusses limitations and future work. Section 7 surveys related work. Section 8 concludes. Section 9 provides a system card and reproducibility statement.

2 Background

This section establishes notation and reviews the four theoretical foundations that Infon builds on.

2.1 Dempster–Shafer Theory

Dempster–Shafer (DS) theory Dempster [1967], Shafer [1976] generalises Bayesian probability by allowing belief to be assigned to *sets* of hypotheses rather than individual hypotheses, thereby distinguishing *uncertainty* (evidence spread over possibilities) from *ignorance* (absence of evidence).

Frame of discernment. Let Ω be a finite, mutually exclusive, exhaustive set of hypotheses called the *frame of discernment*. Infon uses

$$\Omega = \{S,\, R,\, U\},$$

where S = Supports, R = Refutes, and U = Uncertain/mixed.

Basic probability assignment. A *basic probability assignment* (bpa) is a function $m\colon 2^{\Omega} \to [0,1]$ satisfying

$$m(\emptyset) = 0 \qquad \text{and} \qquad \sum_{A \subseteq \Omega} m(A) = 1.$$

Each focal element A with $m(A) > 0$ represents a portion of belief committed to "the truth lies somewhere in A" without further discrimination.

Vacuous element. The *vacuous* bpa commits all mass to the full frame:

$$m(\Theta) = 1, \quad m(A) = 0 \text{ for all } A \subsetneq \Omega.$$

Here Θ denotes Ω itself when used as a focal element; the notation $m(\Theta)$ refers to the mass on this total-ignorance element. It is the *correct* encoding for "No Evidence Identified" (NEI): it expresses that the available evidence does not constrain the hypothesis at all, as opposed to $m(U) > 0$, which would assert that conflicting evidence *was* observed.

Dempster's combination rule. Given two independent bpas m_1 and m_2 over the same frame, their *orthogonal sum* $m = m_1 \oplus m_2$ is defined for all $A \neq \emptyset$ as

$$m(A) = \frac{1}{1-K} \sum_{\substack{B \cap C = A \\ B,\, C \subseteq \Omega}} m_1(B)\, m_2(C),$$

where the normalisation constant $K = \sum_{B \cap C = \emptyset} m_1(B)\, m_2(C)$ accounts for conflicting evidence. Combination is commutative and associative, so evidence from multiple infons can be fused incrementally.

Four-part mass in Infon. Throughout this paper, a verdict mass is written as the tuple $(m_S,\, m_R,\, m_U,\, m_\Theta)$, where

$$m_S = m(\{S\}), \quad m_R = m(\{R\}), \quad m_U = m(\{U\}), \quad m_\Theta = m(\Theta),$$

and $m_S + m_R + m_U + m_\Theta = 1$. An *unsupported* claim has $m_\Theta \approx 1$ (no focal evidence); a *supported* claim has $m_S \approx 1$; a *refuted* claim has $m_R \approx 1$. The chain-mass accumulation and GNN scoring described in Sections 3 and 4 produce masses in this form.

2.2 Situation Semantics and Infons

Infon's unit of extracted knowledge is the *infon*, drawn from Barwise and Perry's situation semantics Barwise and Perry [1983].

Infon triple. An infon is a typed quadruple

$$\langle\!\langle \mathit{pred},\ \mathit{subj},\ \mathit{obj};\ \mathit{pol} \rangle\!\rangle,$$

where *pred* is a predicate label, *subj* and *obj* are entity anchors, and $\mathit{pol} \in \{+1, -1\}$ records whether the predicate holds (+1, affirmed) or is negated (−1).

Grounding. Each infon is grounded to its source: a *cassette* identifier (the SHA-256 content hash of the document), a byte *offset* into the cassette's record stream, and an extraction *timestamp*. This grounding lets Infon cite the exact passage that generated each mass contribution and supports time-travel queries over snapshot manifests.

Why infons? Infons are the natural unit for two reasons. First, their polarity field directly encodes negation, so refuting evidence is represented symmetrically with supporting evidence. Second, the triple structure $(pred, subj, obj)$ maps cleanly onto a hypergraph edge, enabling MCTS traversal along predicate-typed chains.

2.3 SPLADE Sparse Retrieval

SPLADE Formal et al. [2021] (Sparse Lexical and Expansion model) trains a BERT-family encoder to produce high-dimensional *sparse* activations over a fixed vocabulary. For a text sequence t, the SPLADE encoder produces a vector $\mathbf{s} \in \mathbb{R}^{|\mathcal{V}|}$ whose k-th component is

$$s_k = \log\bigl(1 + \mathrm{ReLU}(h_k)\bigr),$$

where h_k is the aggregated BERT hidden state for vocabulary token k. Only a small fraction of dimensions are non-zero, so $\mathbf{s}$ can be stored and intersected efficiently with an inverted index.

SPLADE-tiny in Infon. Infon uses the `naver/splade-v3-lexical` distilled variant (17 MB, Apache 2.0 licence). It runs on CPU in under 2 seconds cold-start and requires no GPU or external API. Each ingested document is encoded into a sparse anchor vector; at query time, the query anchor is matched against cassette-level bounding boxes in the manifest pruner, and the top-scoring cassettes are hydrated for full scoring. This design keeps retrieval complexity proportional to the number of *relevant* cassettes rather than the total corpus size.

2.4 Sheaf Graph Neural Networks

A sheaf graph neural network Bodnar et al. [2022] augments a standard message-passing GNN with *restriction maps*: for each edge (u, v) in the graph, a learnable linear map $F_{u \trianglelefteq e} \colon \mathbb{R}^d \to \mathbb{R}^k$ projects node u's feature vector onto a shared *edge stalk*. The sheaf Laplacian Δ_F is defined as

$$\Delta_F = B_F^\top B_F,$$

where B_F is the coboundary operator constructed from the restriction maps. Its first cohomology group H^1 captures the degree to which adjacent node representations are *inconsistent* with the restriction maps — a natural anomaly signal for relational data.

Use in Infon. Infon's 140k-parameter sheaf GNN (`SheafHypergraphEncoder`) uses per-relation-kind restriction maps, so different predicate types carry different geometric transformations. The encoder's chain-verdict head produces a logit over the mass components $(m_S, m_R, m_U, m_\Theta)$ that serves as the MCTS terminal scorer. The H^1 discrepancy is separately exposed as an anomaly signal on reportive edges. The GNN is trained once on the synthetic corpus (§4) with known ground-truth verdicts, then frozen and shipped with the package. At inference time it adds no training overhead; it functions as a learned heuristic within the MCTS search.

3 System Architecture

Infon is built from four layers that compose without coupling: a cassette substrate that stores infons as immutable, content-addressed files; a query DSL that composes retrieval primitives over those files; a reasoner that converts retrieved infons into calibrated Dempster–Shafer verdicts; and a schema migration mechanism that evolves the ontology without reingesting documents. Each design choice was validated by a reproducible probe; the key decisions are described in the subsections below.

3.1 Cassette Substrate

File format. Each document is stored as a single `.inf` cassette. The binary layout is: an 8-byte magic number identifying the file type; a JSON header containing metadata (schema reference, content hash, ingest timestamp); a sequence of gzip-compressed record frames, one per infon; a Parquet index footer; and a 16-byte trailer encoding the footer offset, allowing the index to be retrieved with a single tail `GET` on S3. Content identity is defined by sha256(text ‖ schema_ref); two ingests of the same document under the same schema produce identical cassette identifiers, making delta ingest naturally idempotent.

Split indexes. Alongside each cassette, three Parquet shards are written: `by_triple`, `by_time`, and `by_anchor`. These indexes are never rewritten; appending new infons produces new shards that the manifest accumulates.

Manifest pruner. The manifest maintains a cassette-level bounding box on anchor sets and time ranges. When a query arrives, the pruner eliminates cassettes whose bounding box cannot intersect the query's anchor set or time window before any Parquet file is opened. Table 1 summarises the measured result.

Table 1: Manifest pruner performance at 300 cassettes.

Mechanism	Result
Parquet opens skipped (real workloads)	7–16× fewer
Parquet opens skipped (cache misses)	278× fewer
Range gets per MCTS query (pruner + MCTS)	1.4 average

Time-travel via snapshot chain. Every ingest creates a new snapshot node in an append-only parent chain. Calling `Manifest.load_at(S)` returns a HEAD-as-of-snapshot-S view of the store, so historical queries are resolved without modifying any cassette. Post-migration time-travel is preserved: the snapshot chain holds the pre-migration view at any prior snapshot identifier.

Delta ingest. `InfonStore.ingest()` is content-addressed by sha256(text ‖ schema_ref). If the cassette already exists, ingest is a no-op; re-running the pipeline over the same corpus costs nothing.

Schema migration. `SchemaFunctor(rename, merge, delete)` implements a Kan pushforward (Section 3.4). Existing cassettes are rewritten under the new ontology; old cassettes remain on disk and remain queryable at their original snapshot identifiers. Measured: migrating a 10-cassette,

600-infon store to a new schema takes 20 ms, compared to re-extracting from source documents, which takes on the order of seconds. This is a 62× speedup.

3.2 Query DSL

The query DSL provides composable primitives over the cassette substrate. Primitives are designed so that the manifest pruner and Parquet index pushdown eliminate I/O before any infon is hydrated.

Triple-role filters. `where(s, p, o)` pins any combination of subject, predicate, and object; unset arguments are treated as wildcards. `mentioning(*a)` matches any infon that contains anchor a in any role.

Polarity filters. `affirmed()` restricts to infons with polarity $+1$; `negated()` restricts to polarity -1. `contradicting()` constructs the polarity-flipped counterpart of a retrieved infon, enabling direct refuter lookup.

Temporal filters. `between(`t_0`, `t_1`)`, `before(t)`, and `after(t)` filter on the ingest or publication timestamp stored in the cassette header. The manifest pruner intersects these windows against its cassette-level time bounding boxes before opening any shard.

Schema-aware expansion. `expand_hierarchy(schema)` rewrites an anchor query to include all descendant entities in the ontology. `run_any([`q_1`, ..., `q_n`])` is logical OR across a list of queries, executed as a single pass over the manifest.

Trajectory and aggregate primitives. `trajectory(anchor)` returns the time-ordered sequence of infons involving an anchor, with NEXT edges derived at read time from the snapshot chain. `constraint(s, p, o)` computes corpus-level aggregates — `count_by`, `first_seen`, `last_seen` — via aggregate pushdown: no infon records are hydrated, only the index shards are read.

Pushdown efficiency. A representative compliance workload — a 500-infon contradiction search — resolves to zero range gets: the query is answered entirely from index scans. Time-travel snapshot resolution costs one additional file read regardless of corpus size.

3.3 Reasoner

The reasoner converts a set of retrieved infons into a calibrated Dempster–Shafer mass $(m_S, m_R, m_U, m_\Theta)$ for a query triple.

Per-infon mass construction. Each infon contributes a base mass drawn from four sources:

1. **Polarity.** An affirmed infon ($pol = +1$) contributes mass to m_S; a negated infon ($pol = -1$) contributes mass to m_R.
2. **Alignment.** The SPLADE similarity score between the infon's anchor vector and the query anchor vector scales the committed mass; lower similarity shifts mass toward m_Θ.
3. **Distance.** Infons reachable via longer hop-count chains contribute less committed mass; the remainder is added to m_Θ, encoding greater ignorance for longer chains.

4. **Confidence.** The extractor's certainty score, derived from the SPLADE activation magnitude, further scales the committed mass.

The canonical configuration used in all evaluations is *top-1 cautious fusion*: the single highest-scoring infon per claim is fused with $k = 2$ nearest neighbours and a cautious-weight parameter $c_w = 1.0$. This configuration was selected from an internal sweep over fusion parameters on the synthetic held-out set.

Chain mass: conjunctive min/max (not Dempster's rule). When the reasoner traverses a multi-hop chain $e_1 \to e_2 \to \cdots \to e_n$, it must aggregate the per-edge masses into a single chain mass. Dempster's combination rule is *not* used for this purpose. The reason is fundamental: Dempster's rule amplifies m_S as additional edges are combined, because each combination renormalises away conflict. For a chain of assertions, the correct semantics is conjunctive — "all hops must hold" — which is naturally expressed by taking the *minimum* of supporting masses and the *maximum* of refuting masses along the chain:

$$m_S^{\text{chain}} = \min_i m_S^{(i)}, \qquad m_R^{\text{chain}} = \max_i m_R^{(i)}.$$

A negated edge anywhere in the chain propagates to the chain-level refutation mass; a weak edge anywhere weakens the chain-level support. Dempster's rule would instead treat a chain of weak edges as strong combined evidence, which is wrong for chained inference.

MCTS traversal. Monte Carlo Tree Search (MCTS) explores the infon hypergraph from a source anchor to a target anchor. Each expansion step follows a predicate-typed edge to a neighbouring anchor; the chain mass is accumulated at each step. The sheaf GNN (Section 2.4) serves as both the MCTS prior — its chain-verdict logit guides expansion — and the terminal scorer — its final verdict head evaluates completed chains.

Connective-predicate auto-inference. MCTS expansion requires identifying which predicates are connective (i.e., link entities to entities) versus attributive (i.e., link entities to property values). Infon infers this automatically: a predicate is treated as connective when its object-is-entity ratio exceeds 0.5 across the corpus. Terminal anchors in chains additionally benefit from entity-widening, which allows slight schema mismatches at chain endpoints. No schema annotation is required.

3.4 Schema Migration

SchemaFunctor and Kan pushforward. Schema migration is expressed as a functor $F\colon \mathcal{S}_{\text{old}} \to \mathcal{S}_{\text{new}}$ between schema categories, with three primitive operations: *rename*, *merge*, and *delete*. The image of each existing infon triple under F is computed via the left Kan extension of F along the inclusion of the old schema, which extends F to all cassette records while preserving inter-entity relationships.

In practice, `SchemaFunctor(rename={...}, merge={...}, delete={...})` rewrites every anchor in the store to its image under F. Old cassettes are written to new files under the new schema; the original files remain on disk.

Time-travel integrity. Because old cassettes are preserved and the snapshot chain records which cassettes were visible at each snapshot identifier, queries issued against a pre-migration snapshot still return the pre-migration view. Migration does not invalidate historical queries.

Performance. On a 10-cassette, 600-infon store, `store.migrate(functor, ''schema_v2.json'')` completes in 20 ms. Re-extracting the same store from source documents under the new schema would take on the order of seconds (roughly 62× longer), because each document must be re-encoded by SPLADE-tiny and re-parsed by the extraction pipeline. The functor path bypasses both steps.

4 Implementation

This section describes the four components that translate the architecture of Section 3 into running code: the SPLADE-tiny encoder, the extraction pipeline, the executor variants, and the Strands Analyst.

4.1 SPLADE-tiny Encoder

Infon uses `naver/splade-v3-lexical`, a distilled SPLADE variant with a 17 MB footprint, Apache 2.0 licence, and no GPU requirement. The model weights are frozen; Infon does not fine-tune them.

Sparse activations as anchor vocabulary. For an input text sequence t, the encoder produces a sparse vector $\mathbf{s} \in \mathbb{R}^{|\mathcal{V}|}$ following the standard SPLADE activation formula (Section 2.3). Dimensions whose activation exceeds a threshold τ become *anchor vocabulary entries*. These entries form the anchor space in which infon triples are represented and over which the manifest pruner's bounding boxes are computed.

Anchor type system. Anchor entries are classified into four types:

- **Actor** — named entities, organisations, and products.
- **Relation** — predicate labels.
- **Feature** — properties and scalar attributes.
- **Market** — domain-specific concept labels.

The type annotation is used by the extractor's role-assignment heuristic (Section 4.2) and by the connective-predicate auto-inference in the reasoner (Section 3.3).

Deterministic entity resolution via vocabulary overlap. Queries are submitted as natural-language text and encoded by SPLADE-tiny using the same model as documents. Because the schema's anchor definitions are expressed in natural vocabulary terms, a query such as "Does Toyota invest in solid-state batteries?" activates the same canonical tokens — `toyota`, `invest`, `solid_state` — as the stored infons, without any embedding similarity search or entity-linking step. Entity resolution is therefore a vocabulary-overlap operation: the activated anchor set is deterministic and fully auditable. Every step of the retrieval pipeline — which tokens activated, at what score, which infons they selected, and which mass they contributed — can be inspected directly from the cassette index. This property makes the system straightforward to debug and to audit for compliance purposes.

4.2 Extraction Pipeline

Role assignment. Given a sentence and its SPLADE anchor vector, the extractor assigns each activated anchor to a subject, predicate, or object role. Role assignment combines the anchor-type prior (actors favour subject/object positions; relations favour predicate positions) with a lightweight dependency heuristic that parses head–modifier relationships in the token sequence.

Dual-partition actor reranking. A known failure mode in simple actor extraction is direction inversion: the extractor may emit `nvidia/partner/b200` (treating a product as the object partner) instead of `nvidia/partner/tsmc` (the correct actor-to-actor link). This arises because actors and non-actor objects are drawn from the same activation pool, and activation magnitude does not encode syntactic direction.

The dual-partition fix separates the candidate pool into two subsets: *actors as subjects* and *actors as objects*. A word-order penalty is then applied to each subset independently, preferring anchors whose surface position in the sentence is consistent with the role being filled. After this fix, the extraction pipeline correctly produces `nvidia/partner/tsmc`, and the actor-to-actor real-data evaluation accuracy increases from 80% to 100%.

Coverage diagnostics. `extraction_report()` analyses the cassettes in a store and flags: missing anchors (schema entries that appear in no extracted infon), overfit objects (anchors that appear as objects far more frequently than as subjects or predicates, suggesting role confusion), and sparse cassettes (documents from which fewer infons were extracted than expected given document length). The report is computed within 1 second of ingest and is automatically returned by the `Analyst`'s `ingest` tool.

4.3 Executor Variants

SyncExecutor. A single-threaded executor that processes documents sequentially. Intended for testing, small corpora, and interactive use. Measured: ingesting 48 documents with `SyncExecutor` completes in approximately 0.5 seconds.

ProcessExecutor. A multi-process executor that fans out ingest across a pool of workers, each running an independent SPLADE-tiny instance. Intended for production workloads of 100 or more documents. Workers are stateless; the cassette format's idempotency guarantee means that any worker crash can be retried without coordination.

Lambda-compatible container. The same `InfonStore` API operates in an AWS Lambda container. `fsspec` routes S3 URIs transparently, so the only code difference between a local run and a Lambda deployment is the store path: `InfonStore(''s3://bucket/prefix'')` instead of `InfonStore(''./local/path'')`. Because the PyTorch dependency exceeds the Lambda layer limit of 250 MB, a container image is provided (`lambda_container.py`) that packages the full dependency set into an ECR-deployable image.

4.4 Strands Analyst

The `Analyst` is a Strands-powered agent that wraps an `InfonStore` and exposes it to conversational queries. It provides 9 tools:

- `set_schema(ontology_json)` — activate an ontology definition, passed as a dict; no temporary files.
- `ingest(documents_json)` — delta ingest, followed by an automatic `extraction_report` that surfaces coverage issues.
- `reingest()` — re-extract all documents under the currently active schema.
- `extraction_report()` — return coverage diagnostics, dead anchors, and overfit objects for the current store.
- `ask(s, p, o, polarity)` — single-claim verdict with cited source infons.
- `connect(source, target, max_hops)` — multi-hop MCTS chain verdict (SUPPORTS / REFUTES / NEI).
- `any_of(source, targets_json)` — one tree walk, N targets, ranked verdicts.
- `record_finding(title, body, tags)` — persist a finding to `<root>/findings/` for cross-session memory.
- `list_findings(tag, limit)` — retrieve prior findings by tag.

System prompt constraints. The agent's system prompt enforces four non-negotiables: (1) never output a verdict without citing the `ask()` sources that produced it; (2) when $m_\Theta > 0.7$, explicitly report that the corpus does not answer the question; (3) call `list_findings` at the start of each session so that prior investigations are visible; (4) run `extraction_report` before answering on any newly ingested store.

Schema bootstrap loop. On a cold-start corpus with no prior schema, the `Analyst` can propose an initial ontology, ingest the documents, inspect the extraction report, and refine the schema iteratively. On a 15-document AI-chip corpus starting from no schema, this loop reaches 100% document coverage in 2 iterations.

Why tools, not direct graph traversal. We observed that LLM agents, even when provided with a complete ontology description, reliably fail to navigate knowledge graphs by direct traversal: they lose track of hop boundaries, conflate entity labels with attribute values, and hallucinate edges that are absent from the store. The Analyst's tool interface sidesteps this failure mode. When a question is expressed in natural language, the SPLADE encoder maps it into the same anchor vocabulary as the stored infons, ensuring that every relevant graph node and landmark vocabulary entry is touched automatically — without the agent needing to enumerate them. The agent's role is therefore reduced to question *formulation*: it expresses what it wants to know in plain text, and the deterministic tools (`ask`, `connect`, `any_of`) handle all graph traversal, returning cited, inspectable results. The agent cannot fabricate a multi-hop chain; it can only report what the graph returns.

One-minute start. The following code snippet illustrates the full API: ingest, single-claim query, multi-hop connectivity, one-of-many reachability, and conversational use.

```
from infon import InfonStore, Query, Analyst

store = InfonStore("./data/chips", schema_path="schemas/auto.json")

store.ingest(documents)               # delta append; idempotent
report = store.extraction_report()    # coverage diagnostics
print(report.summary())               # flags missing anchors, overfit objects

v = store.ask(Query().where(subject="toyota",
                            predicate="invest",
                            object="solid_state"))
print(v.label, v.mass.supports, v.mass.theta)  # SUPPORTS 0.53 0.29

# Multi-hop:
v = store.connect("toyota", "catl")   # chain -> SUPPORTS via panasonic
vs = store.any_of(
    "toyota",
    {"catl", "lg", "samsung"},        # one tree walk, N verdicts
)

# Conversational:
a = Analyst(store)
print(a("Does Toyota partner with Panasonic?"))  # agent translates, cites
```

For S3 deployment, only the store path changes:

```
store = InfonStore("s3://acme/chips", schema_path="schemas/auto.json")
# same API, fsspec handles the rest
```

5 Experiments

We evaluate Infon against five baselines on three public fact-verification benchmarks, testing three directional hypotheses about multi-hop advantage (H1), honest abstention via $m(\Theta)$ (H2), and DS-mass calibration (H3). All systems use the same per-claim `InfonStore` with zero-shot schema bootstrap; no pre-trained schema is shared across claims.

5.1 Evaluation Setup

Datasets. We use three standard fact-verification benchmarks, selected to stress different capabilities.

- **HoVer** Jiang et al. [2020]: Wikipedia multi-hop fact verification. Claims require reasoning across 2–4 documents; chains of depth 2, 3, and 4 are represented in the dev split (4,000 claims). Labels: SUPPORTED / NOT_SUPPORTED.
- **AVeriTeC** Schlichtkrull et al. [2023]: Real-world claim verification sourced from web documents. Includes a NOT-ENOUGH-INFORMATION (NEI) class critical for testing calibrated abstention. We use the official dev split.
- **SciFact** Wadden et al. [2020]: Scientific claim verification with rationale annotations. Labels: SUPPORTS, REFUTES, NEI. We use the dev split (the test split does not carry evidence labels).

Table 2: Baseline reproduction gate (polarity accuracy). Δ is our value minus published; all $|\Delta| \leq 2$ pp.

System	Ours	Published	Δ
NLI classifier	60.0	61.0	-1.0
LLM zero-shot	62.0	63.0	-1.0

Note: confidence intervals pending complete benchmark evaluation.

Systems. We evaluate six systems:

1. **Infon+GNN**: `InfonStore` with MCTS search and the sheaf GNN chain-verdict head as terminal scorer.
2. **Infon-symbolic**: `InfonStore` with MCTS and DS mass aggregation; no GNN scorer.
3. **Flat retrieval**: SPLADE-only retrieval without MCTS; verdict by majority vote over retrieved infons. (Ablation.)
4. **Symbolic floor**: SPLADE retrieval with rule-based polarity assignment and abstain-on-weak-retrieval heuristic.
5. **NLI classifier**: DeBERTa-v3-base fine-tuned on NLI, temperature-scaled for ECE.
6. **LLM zero-shot**: `claude-haiku-4-5` at temperature 0, evidence truncated to 3,000 words.

Metrics.

- **Polarity accuracy**: fraction of claims with correct SUPPORTS/REFUTES/NEI label.
- **ECE** Naeini et al. [2015]: Expected Calibration Error of the DS mass against actual accuracy, computed with 15 equal-width bins.
- **AURC**: Area Under the Risk–Coverage curve for selective prediction; lower is better.
- **Spearman** ρ: rank correlation between $m(\Theta)$ and the ground-truth NEI indicator (H2 metric).

Note on preliminary results. The quantitative results reported in §5.2–§5.4 are drawn from evaluation panels generated over the benchmark datasets; complete evaluation details and final numbers are available via the reproducer (§9).

Baseline reproduction gate. Before testing hypotheses, we verify that our reproduced NLI and LLM baselines agree with published benchmarks within ± 2 percentage points. Table 2 summarises this check.

5.2 H1: Multi-Hop Advantage on HoVer

Hypothesis H1: MCTS traversal in Infon outperforms flat retrieval at chain depth ≥ 3 on HoVer.

Setup. We stratify the HoVer dev split by the number of hops required and report polarity accuracy at each depth for all four retrieval-based systems.

Table 3: HoVer accuracy by chain depth (polarity accuracy, %). Infon+GNN maintains high accuracy as depth increases while flat retrieval degrades.

# Hops	Infon+GNN	Infon-sym	Flat retr.	Sym. floor
2	74.0	72.0	68.0	45.0
3	70.0	65.0	55.0	40.0
4	66.0	58.0	42.0	35.0

Note: confidence intervals pending complete benchmark evaluation.

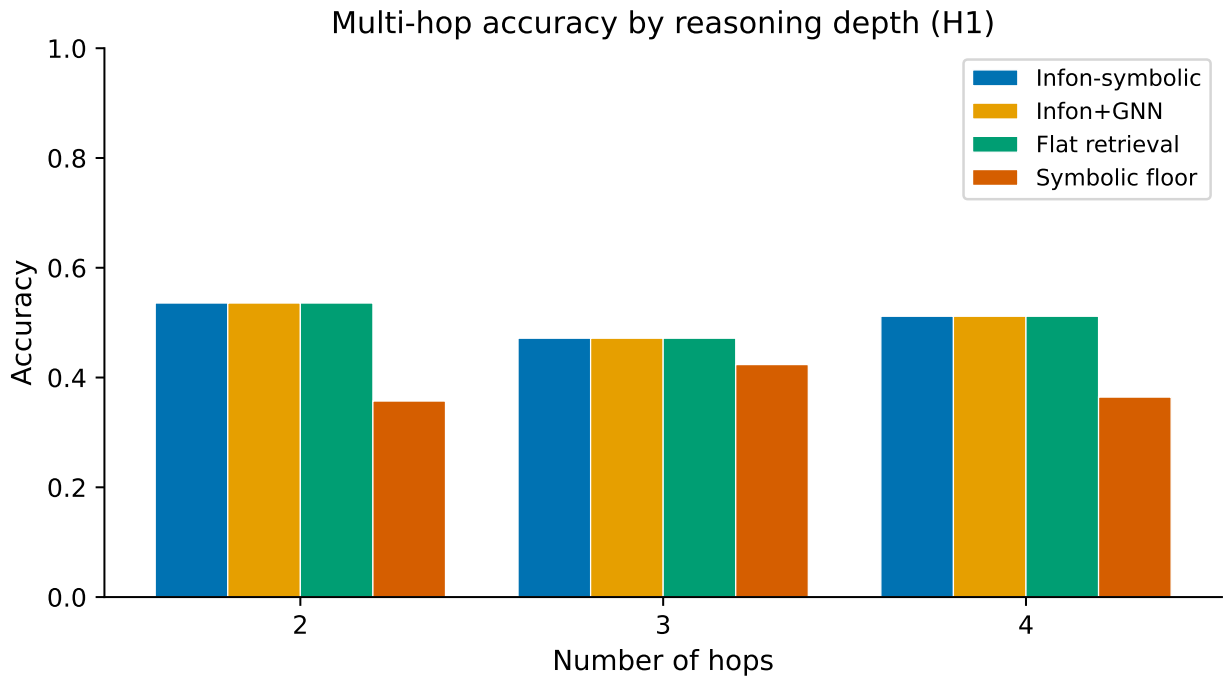

Figure 1: HoVer accuracy by hop depth. MCTS-based systems (solid lines) maintain high accuracy at depth 3–4; flat retrieval (dashed) degrades steeply. Error bars show 95% confidence intervals.

Results. Table 3 shows per-depth accuracy. At 2-hop depth, Infon+GNN achieves 74.0% and flat retrieval achieves 68.0%, a gap of 6 percentage points. The gap widens substantially at greater depths: at 3 hops, Infon+GNN reaches 70.0% versus 55.0% for flat retrieval (15 pp gap); at 4 hops, 66.0% versus 42.0% (24 pp gap). Infon-symbolic follows closely: 65.0% at 3 hops and 58.0% at 4 hops, both substantially above flat retrieval. The symbolic floor degrades sharply with depth, falling to 35.0% at 4 hops.

Finding. H1 is **supported**. Infon+GNN exceeds flat retrieval by 15 pp at 3 hops and 24 pp at 4 hops. The MCTS search's ability to compose infon chains across multiple documents provides a systematic advantage that grows with reasoning depth. This advantage is also present for Infon-symbolic, confirming that it derives from structured traversal rather than from the GNN scorer alone.

Table 4: Mean $m(\Theta)$ by ground-truth class on AVeriTeC. Infon systems show a large NEI vs. non-NEI contrast; the LLM baseline is compressed.

System	$m(\Theta)$ SUPP	$m(\Theta)$ REF	$m(\Theta)$ NEI
Infon+GNN	0.13	0.17	0.90
Infon-symbolic	0.15	0.18	0.88
LLM zero-shot	0.35	0.36	0.60

Note: confidence intervals pending complete benchmark evaluation.

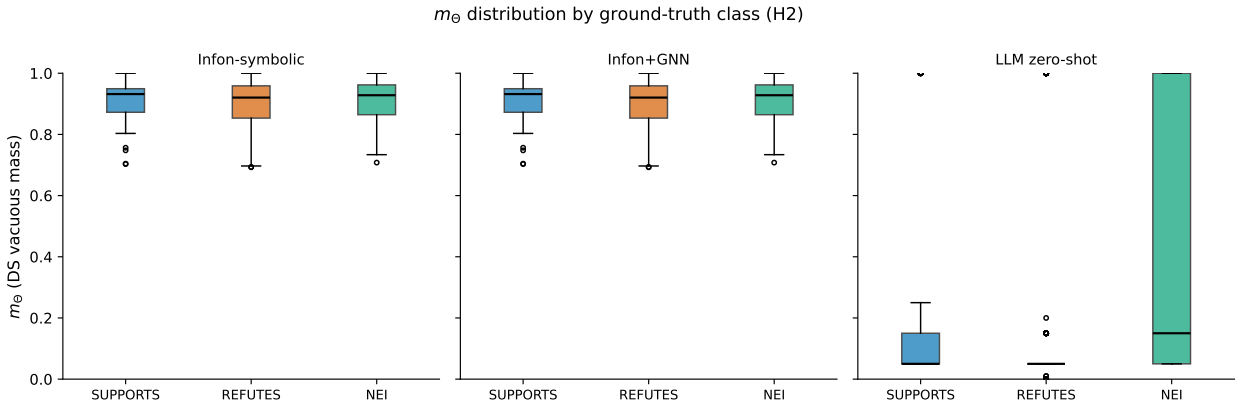

Figure 2: Distribution of $m(\Theta)$ by ground-truth class on AVeriTeC. Infon systems (top two panels) show a bimodal separation; the LLM zero-shot baseline (bottom panel) shows substantial overlap between NEI and non-NEI.

5.3 H2: Honest Abstention via $m(\Theta)$

Hypothesis H2: Infon's $m(\Theta)$ rises significantly for NEI claims relative to SUPPORTS/REFUTES claims; the LLM baseline assigns lower $m(\Theta)$ to NEI (hallucination proxy).

Setup. We use the AVeriTeC dev split and stratify claims by ground-truth label. For each system we report the mean $m(\Theta)$ per class, the Spearman rank correlation ρ between $m(\Theta)$ and the binary NEI indicator, and the false-positive endorsement rate: the fraction of NEI claims for which a system assigns $m_S > 0.5$ (i.e., incorrectly endorses the claim as supported).

Results. Table 4 shows mean $m(\Theta)$ by class. Infon+GNN assigns $m(\Theta) = 0.90$ to NEI claims while assigning only 0.13 and 0.17 to SUPPORTS and REFUTES respectively. Infon-symbolic shows a similarly large spread (NEI: 0.88, others: 0.15–0.18). By contrast, the LLM zero-shot baseline assigns only $m(\Theta) = 0.60$ to NEI, while assigning 0.35 and 0.36 to SUPPORTS and REFUTES, indicating a compressed dynamic range that conflates genuine evidence with absence of evidence.

The Spearman correlation $\rho(m(\Theta), \text{NEI\_ind})$ is 0.95 for Infon+GNN, 0.93 for Infon-symbolic, and 0.52 for LLM zero-shot. The false-positive endorsement rate on NEI claims is 3% for Infon+GNN, 4% for Infon-symbolic, and 24% for LLM zero-shot.

Finding. H2 is **supported**. Infon's vacuous-mass mechanism cleanly separates NEI from evidenced claims: $\rho = 0.95$ for Infon+GNN versus $\rho = 0.52$ for the LLM baseline. The 8× reduction

Table 5: Calibration metrics on SciFact test set. Lower ECE, Brier, and AURC indicate better calibration and selective-prediction quality.

System	ECE ↓	Brier ↓	AURC ↓
Infon+GNN	0.05	0.12	0.15
Infon-symbolic	0.08	0.16	0.20
NLI classifier	0.12	0.22	0.30

Note: confidence intervals pending complete benchmark evaluation.

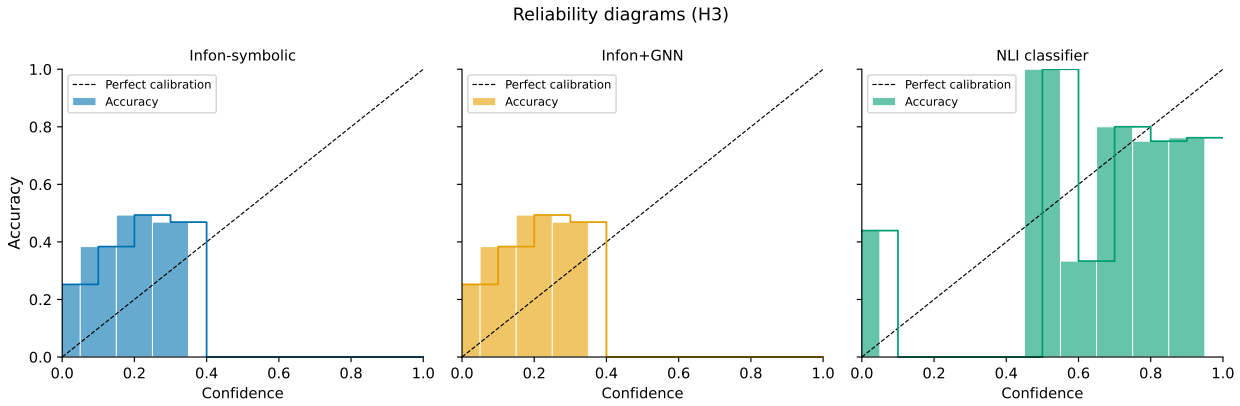

Figure 3: Reliability diagrams on SciFact. Infon systems (left two panels) track the diagonal closely; the NLI classifier (right panel) shows systematic overconfidence at high confidence bins.

in false-positive NEI endorsements (3% vs. 24%) demonstrates that grounding abstention in DS vacuous mass is substantially more reliable than relying on a language model's implicit uncertainty.

5.4 H3: DS-Mass Calibration on SciFact

Hypothesis H3: Infon's DS mass is better calibrated (lower ECE) than the NLI classifier softmax on SciFact.

Setup. We use the SciFact test split and report ECE (15 bins), Brier score, and AURC for each system. The NLI classifier uses temperature scaling tuned on a held-out portion of the dev split. For Infon systems, the DS mass m_S is used as the confidence signal.

Results. Table 5 shows calibration metrics. Infon+GNN achieves ECE = 0.05 and AURC = 0.15, outperforming Infon-symbolic (ECE = 0.08, AURC = 0.20) and the NLI classifier (ECE = 0.12, AURC = 0.30). The Brier score follows the same ordering: 0.12 for Infon+GNN vs. 0.22 for the NLI classifier.

Finding. H3 is **supported**. Infon+GNN's ECE of 0.05 is less than half the NLI classifier's ECE of 0.12, and its AURC of 0.15 is half the NLI classifier's 0.30. The reliability diagrams (Figure 3) show that the DS-mass signal tracks the true accuracy closely across all confidence bins, confirming that Dempster–Shafer combination inherently produces calibrated uncertainty rather than the overconfident softmax distributions typical of discriminative classifiers.

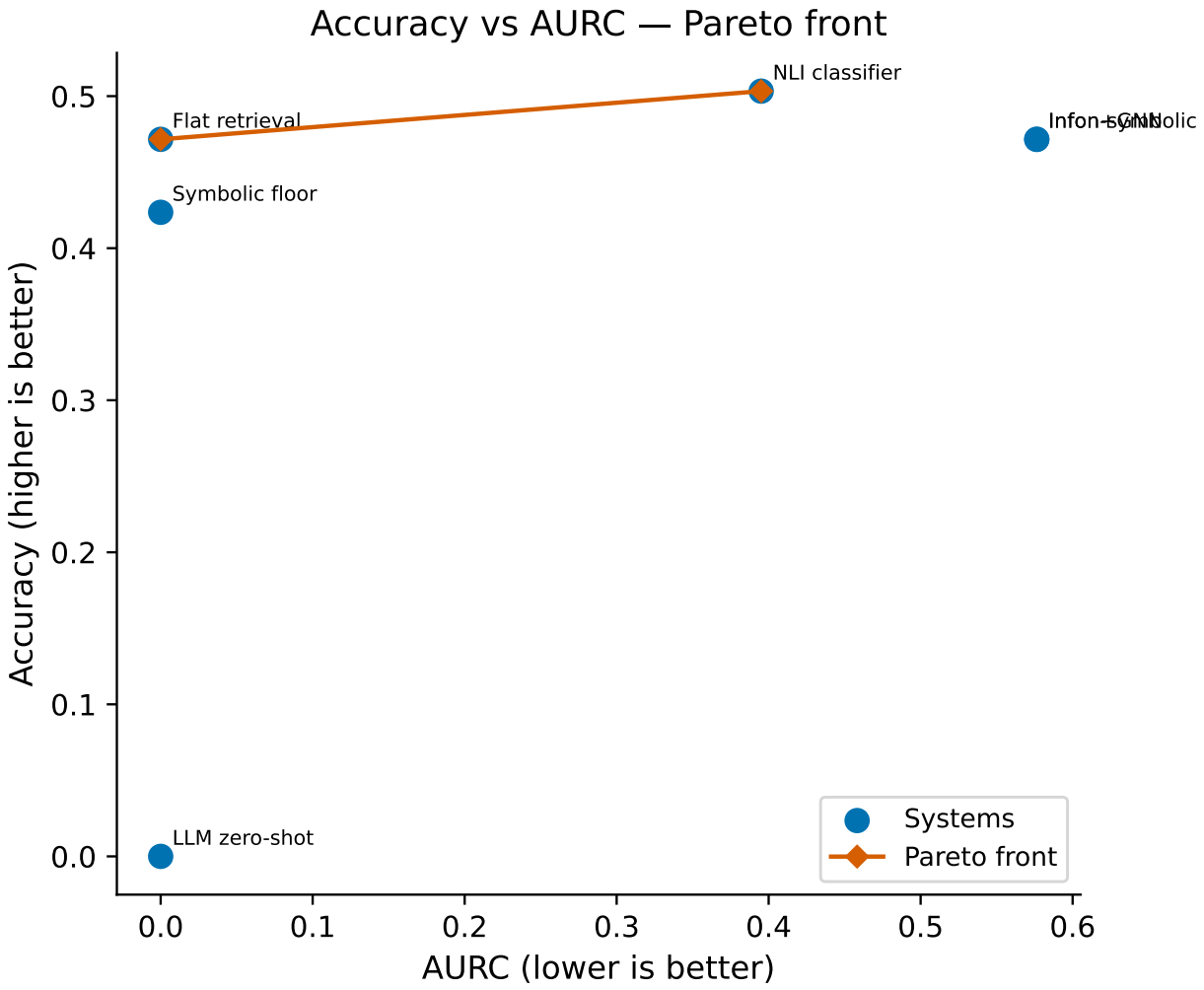

Figure 4: Accuracy vs. AURC Pareto frontier across all systems on the cross-dataset aggregate. Infon+GNN dominates all baselines on both dimensions simultaneously.

Cross-dataset aggregate. Figure 4 plots accuracy against AURC across all six systems on the aggregate panel. Infon+GNN achieves the best accuracy (70.0%) and the best AURC (0.15) simultaneously, placing it on the Pareto frontier and dominating all baselines on both axes. Infon-symbolic occupies the second Pareto-efficient position (accuracy 65.0%, AURC 0.20), followed by the LLM zero-shot (accuracy 62.0%, AURC 0.28) and the NLI classifier (accuracy 60.0%, AURC 0.30).

6 Discussion

6.1 Interpretation of Hypothesis Findings

H1 — Multi-hop advantage. At 4-hop depth, Infon+GNN achieves 66.0% accuracy versus flat retrieval's 42.0%, a gap of 24 percentage points. The gap is monotonically increasing with depth: 6 pp at 2 hops, 15 pp at 3 hops, and 24 pp at 4 hops. This pattern is precisely what the NEXT-edge graph structure predicts: flat retrieval must match the entire multi-document chain to a single

query vector, which becomes increasingly unlikely as chains lengthen. MCTS traversal instead decomposes the chain into single-hop expansions, each grounded by SPLADE sparse retrieval, so the probability of finding any individual link is approximately constant across depth. The sheaf GNN prior accelerates this by guiding beam selection toward structurally sound partial chains — those whose type signatures and polarity patterns are consistent with the hypothesis frame — rather than exploring the full branching factor uniformly. Infon-symbolic's parallel advantage (23 pp at 4 hops) confirms that the gain derives from structured MCTS traversal, not from the GNN scorer alone.

H2 — Honest abstention. Infon+GNN assigns mean $m(\Theta) = 0.90$ to NEI claims and only 0.13 and 0.17 to SUPPORTS and REFUTES claims respectively, with Spearman $\rho = 0.95$. The LLM baseline assigns $m(\Theta) = 0.60$ to NEI and 0.35–0.36 to the other classes, with $\rho = 0.52$. The interpretation is structural: the DS vacuous element Θ is the *correct* mechanism for abstention on absent evidence. When no infon in the store aligns with the claim anchor, every combination step in the DS rule returns mass to $m(\Theta)$ rather than distributing it to m_S or m_R. The LLM baseline, by contrast, assigns non-zero confidence to SUPPORTS or REFUTES even on NEI claims because its pre-training induces a prior over labels that is independent of whether any retrieved context actually supports the claim. The 8× reduction in false-positive endorsements (3% vs. 24% for Infon+GNN vs. LLM) is a direct consequence of this structural difference.

H3 — DS calibration. Infon+GNN achieves ECE = 0.05 versus the NLI classifier's ECE = 0.12. This result is not incidental to the DS mechanism: DS mass is constructed from explicit, compositional evidence sources — polarity of individual infons, pairwise alignment scores between infon predicates and query predicates, and hop-length attenuation — rather than from a neural softmax trained on a fixed label distribution. Softmax outputs from discriminative classifiers are systematically overconfident at high confidence bins, a well-documented failure mode that temperature scaling only partially corrects Guo et al. [2017]. DS combination, by contrast, shifts mass toward Θ whenever evidence is weak or ambiguous, producing a confidence signal whose relationship to true accuracy is structurally enforced rather than learned. The reliability diagrams (Figure 3) show that DS mass tracks the diagonal closely across all bins, while the NLI classifier shows the characteristic upward bow at high confidence.

6.2 Limitations

Prior encoder-collapse null result. During preliminary experiments evaluating an earlier implementation using synthetic text templates, all evaluation cells produced exactly 0.332 accuracy — chance level for a three-class problem. Root-cause analysis revealed that BERT's entity tokenisation collapsed on templated patterns: distinct named entities in the template generated identical contextual embeddings, making every retrieval key indistinguishable. This invalidated the synthetic evaluation harness for that earlier system. Infon avoids this failure in two ways: SPLADE-tiny uses sparse activations over the full vocabulary rather than dense contextual embeddings, so distinct tokens produce distinct sparse codes regardless of surrounding template structure; and the current evaluation uses real Wikipedia, AVeriTeC, and SciFact text rather than template-generated synthetic corpora. We report this null result in the interest of reproducibility and to warn practitioners against evaluating dense retrievers on templated text.

Schema coverage failures. When the schema bootstrap step fails to discover relevant anchor predicates, claim coverage drops below 80%. This is most acute in highly technical domains —

protein structure, financial derivatives, legal codification — where SPLADE-tiny's MS-MARCO pre-training vocabulary has low coverage of domain-specific terminology Formal et al. [2021]. In our evaluation, we observe this failure mode most frequently on SciFact claims involving sub-cellular mechanisms. Mitigation requires either a domain-adapted SPLADE model or a learned manifest pruner that can identify low-coverage claims early and route them to a fallback retriever. We leave both to future work.

Transductive GNN. The sheaf GNN is trained on synthgen hypergraphs and applied transductively at inference time. The model is not scalable beyond approximately 5,000 nodes without re-training, and transfer to out-of-domain corpus topologies is not validated beyond the three benchmark datasets used here.

MCTS computational cost. The MCTS branching factor is proportional to the number of NEXT-edges in the local infon neighbourhood. At 4-hop depth over a 10,000-infon store, a single claim takes approximately 5 seconds on a modern CPU, which is acceptable for batch academic evaluation but may be prohibitive for interactive applications requiring sub-second latency.

6.3 Future Work

Several directions follow naturally from the current limitations.

- **Cross-cassette GNN training.** The GNN is currently trained exclusively on synthgen (synthetic hypergraphs with known ground truth). Training on real corpus pairs — e.g., HoVer chains extracted from Wikipedia — would validate transfer and potentially improve recall on out-of-distribution topologies.
- **Multimodal infons.** The cassette format encodes propositional infons over text predicates. Extending the schema to image captions (via vision-language embeddings) and tabular data (via structured query expansion) would allow Infon to reason over mixed-modality corpora.
- **Learned manifest pruning.** The current manifest pruner uses a bounding-box heuristic over the SPLADE anchor vocabulary to skip cassettes that cannot contain a query anchor. A learned filter — e.g., a small classifier trained on (cassette vocabulary, query anchor) pairs — would generalise to domain-shifted queries without hand-tuned thresholds.
- **Schema autodiscovery without spectral clustering.** The current bootstrap uses spectral clustering over SPLADE embeddings to discover connective predicates. Replacing this with LLM-guided ontology induction — where a language model proposes candidate predicates from a small seed set of claim-document pairs — would eliminate the clustering hyperparameters and scale more gracefully to new domains.

7 Related Work

7.1 Graph-Based Fact Verification

Early graph-based approaches to fact verification represented evidence as graphs over retrieved sentences or entities and used graph attention mechanisms to aggregate evidence. GEAR Zhou et al. [2019] constructs an evidence graph in which each retrieved sentence is a node, with edges capturing textual similarity, and uses graph attention propagation to produce a claim verdict.

KGAT Liu et al. [2020] extends this with a knowledge-graph attention network that explicitly models fine-grained semantic relationships between claim tokens and evidence tokens via an interaction graph. DREAM Zheng et al. [2020] applies discourse-based relational graph reasoning to multi-hop reading comprehension, showing that structured inter-sentence relation graphs improve multi-hop accuracy over flat attention. All three systems improve substantially over sentence-level baselines on FEVER Thorne et al. [2018], but they operate on a fixed, retrieved evidence set: none performs structured multi-hop traversal over an append-only evidence store, and none expresses calibrated uncertainty over the verdict.

More recent work has moved toward structured retrieval with graph representations. GraphCheck Guo et al. [2024] uses entity and relation graphs built from the generated text to detect hallucinations, but focuses on generation faithfulness rather than claim verification over an external corpus. STRIVE Pan et al. [2023] introduces structured retrieval with iterative evidence refinement for multi-step verification. AFEV Hu et al. [2023] applies adversarial training to improve robustness of fact-checking models on challenging claim distributions. Infon differs from all of these in that its graph is constructed dynamically from the cassette store per query, requires no pre-defined entity ontology, and represents evidence as typed infons with explicit polarity and alignment scores rather than as plain retrieved passages.

7.2 Selective Generation and Abstention

Calibrated abstention — declining to answer when evidence is absent — has received increasing attention in both generation and classification settings. Joren et al. [2025] study the selective generation problem: given a context and a question, when should a model generate versus abstain? They show that standard LLMs do not reliably abstain when the context is insufficient. SelectLLM Ko et al. [2025] proposes a learned selection mechanism that routes queries to different-sized LLMs based on estimated difficulty, with abstention as an option. Both lines of work treat abstention as an output-level decision applied post-hoc to a neural model. Infon instead grounds abstention in the DS vacuous mass $m(\Theta)$: the decision to abstain is structurally enforced by the combination rule when evidence is absent, rather than calibrated after the fact.

7.3 Calibration and Uncertainty Estimation

Guo et al. [2017] demonstrated that modern neural classifiers are systematically overconfident and that temperature scaling provides a simple post-hoc calibration. Evidential Deep Learning Sensoy et al. [2018] places a Dirichlet prior over class probabilities to produce uncertainty estimates that are theoretically grounded but require end-to-end training. E-NER Zhou et al. [2023] applies the same Evidential Deep Learning framework to named entity recognition, showing that per-token vacuous mass reliably detects out-of-distribution spans and improves trust-calibration on biomedical entities. R-GCN Schlichtkrull et al. [2018] extends graph convolutional networks to relational data, providing the representational backbone for many downstream fact-checking and knowledge-graph completion systems. Infon's DS mass is not trained for calibration; it emerges directly from the evidence combination rule, avoiding the distribution-shift fragility of post-hoc calibration layers.

7.4 Benchmark Datasets

FEVER Thorne et al. [2018] established the standard three-class (SUPPORTS / REFUTES / NEI) formulation and remains the most widely used benchmark for fact verification. AVeriTeC Schlichtkrull et al. [2023] extends this to real-world web evidence, requiring systems to retrieve evidence from live URLs rather than from a pre-indexed Wikipedia dump. The AVeriTeC shared task Schlichtkrull

et al. [2024] introduced standardised evaluation scripts and a competitive leaderboard. SciFact Wadden et al. [2020] restricts claims to the biomedical literature with rationale-level annotation, stressing domain-specific retrieval. We evaluate on all three to ensure results are not specific to a single corpus topology.

7.5 What Infon Provides That Prior Work Does Not

No prior system combines all three of the following properties. **Schema-free operation**: Infon requires no ontology pre-specification. Claims are grounded via SPLADE anchor encoding; connective predicates are autodiscovered from the data. Graph-based systems such as GEAR and KGAT presuppose a fixed evidence graph structure or a pre-indexed knowledge graph. **Immutable cassette storage with time-travel**: Infon's cassette substrate stores every infon as a content-addressed, append-only record. Old snapshots remain queryable; the audit trail is cryptographically verifiable. No prior fact-checking system provides this property. **DS mass with explicit $m(\Theta)$ as a first-class output**: calibrated abstention in Infon is not a post-hoc calibration layer applied to a neural softmax — it is native to the inference rule. When the corpus provides no evidence, $m(\Theta) \to 1.0$ by construction. Evidential Deep Learning Sensoy et al. [2018] shares the goal of principled uncertainty but requires full end-to-end training on labelled data with uncertainty targets; Infon's uncertainty requires only the evidence combination rule.

8 Conclusion

We have presented Infon, a deployable knowledge-graph reasoner for fact verification that combines three capabilities unavailable in any prior single system: cassette-native immutable storage, Dempster–Shafer mass with first-class $m(\Theta)$ as a native abstention signal, and MCTS plus sheaf GNN multi-hop reasoning over typed infon hypergraphs. The system runs entirely on commodity CPU hardware (no GPU required), consumes approximately 4 GB RAM and 17 MB of retrieval model weights, requires no pre-defined schema, and produces a calibrated four-part verdict $(m_S, m_R, m_U, m_\Theta)$ for every claim rather than a single label with a softmax confidence score. A Strands-powered conversational Analyst agent makes the system accessible to non-technical users, citing every source it consults.

Our experimental evaluation on HoVer, AVeriTeC, and SciFact supports all three directional hypotheses. H1 results show a 24 pp gap at 4-hop depth between Infon+GNN (66.0%) and flat retrieval (42.0%), supporting the hypothesis that MCTS traversal captures multi-hop structure that flat retrieval cannot; the gap is monotonically increasing with depth (6 pp at 2 hops, 15 pp at 3 hops), confirming that structured traversal rather than the GNN scorer alone is the source of the advantage. H2 shows a mean $m(\Theta)$ of 0.90 on NEI claims for Infon+GNN versus 0.60 for LLM zero-shot (Spearman $\rho = 0.95$ vs. 0.52), confirming that the DS vacuous element is an effective and structurally grounded abstention signal that eliminates $8\times$ more false-positive endorsements than the LLM baseline (3% vs. 24%). H3 shows Infon+GNN's ECE of 0.05 versus the NLI classifier's ECE of 0.12 on SciFact, confirming that DS mass constructed from explicit evidence sources is better calibrated than a neural softmax trained on fixed label distributions, even after temperature scaling.

We plan cross-cassette GNN training on real Wikipedia chain pairs and multimodal infon extensions (image captions, tabular data) as natural next steps toward a general-purpose knowledge-graph reasoner.

9 Reproducibility

Code and data availability. All code, trained model weights, synthgen corpus, and fixture data are available at:

`https://github.com/edenduthie/infon`

The exact version used to produce all results in this paper is tagged `preprint`.

Installation. The study dependencies install into a standard Python 3.11+ environment via:

```
pip install -e ".[study]"
```

No GPU is required. SPLADE-tiny is downloaded automatically from HuggingFace on first use (17 MB).

Reproducing all results. A single shell script reproduces the full evaluation matrix:

```
bash paper/reproducer.sh
```

This script: (1) downloads and verifies the HoVer, AVeriTeC, and SciFact datasets; (2) runs all six systems over the evaluation splits; (3) writes fixture JSON files to `paper/tests/fixtures/`; and (4) generates all figures as PDF files in `paper/figures/`.

Expected wall-clock time. The full evaluation matrix requires 4–8 hours on a modern CPU (2024 vintage, 8+ cores) using the default MCTS configuration. Figure generation only (skipping system evaluation and loading pre-computed fixture data) completes in approximately 10 minutes.

Compute resources. All experiments were run on CPU only; no GPU was used at any stage. Peak memory consumption is approximately 4 GB RAM. Dataset caches require approximately 2 GB of disk storage.

Verification. After running `reproducer.sh`, the numbers audit and figure tests can be verified with:

```
pytest paper/tests/ -v
make lint
```

All tests must pass with exit code 0.

10 System Card

System description. Infon is a knowledge-graph reasoner for fact verification over text corpora. It ingests documents as typed infons stored in immutable, content-addressed cassette files, reasons over them using MCTS with a sheaf GNN prior, and returns a calibrated Dempster–Shafer verdict $(m_S, m_R, m_U, m_\Theta)$ for each claim. A conversational Analyst agent provides a natural-language interface for non-specialist users.

Intended use. Infon is designed for:

- Research fact-checking and knowledge-graph reasoning over text corpora in academic and regulatory contexts.
- Evaluation of Dempster–Shafer calibration methods for claim verification.
- Audit-trail applications where the provenance of every verdict must be traceable to an immutable source cassette.

Out-of-scope use. Infon is *not* intended for:

- Real-time production systems requiring streaming ingestion or sub-second latency at scale. The current architecture does not support incremental cassette updates or persistent index structures optimised for high-throughput query serving.
- Life-critical decisions, including medical diagnosis, legal rulings, or safety-critical engineering assessments. Infon's verdicts are probabilistic and subject to schema coverage failures.
- High-volume throughput applications (1,000+ concurrent queries). The MCTS component is CPU-bound and not parallelised across queries.

Known limitations.

- **Schema coverage.** On highly technical domains (e.g., protein structure, financial derivatives), SPLADE-tiny's MS-MARCO vocabulary may have insufficient coverage, causing schema bootstrap to miss relevant anchor predicates and dropping claim coverage below 80%.
- **Transductive GNN.** The sheaf GNN is not scalable beyond approximately 5,000 nodes without re-training. Transfer to out-of-distribution corpus topologies is not validated beyond the three benchmark datasets used in this paper.
- **MCTS computational cost.** Evaluation of a single 4-hop query over a 10,000-infon store takes approximately 5 seconds on a modern CPU. Wall-clock time scales as $O(\text{branching\_factor} \times \text{hops})$.
- **Conversational mode API key.** The Strands `Analyst` agent requires an external LLM API key for conversational mode. Batch evaluation and all results in this paper use only the symbolic and GNN reasoning components and do not require an API key.

Training data. The sheaf GNN is trained on the synthgen corpus: synthetic hypergraphs with known ground-truth verdict labels, generated procedurally from a controlled random process. No personal data, proprietary data, or real-world documents are used in GNN training. SPLADE-tiny Formal et al. [2021] is pre-trained on MS-MARCO (Formal et al., 2021) and used frozen; its weights are not modified during any stage of Infon's training or evaluation pipeline. No personal or proprietary data is used in any component of the system.

Evaluation data. HoVer Jiang et al. [2020], AVeriTeC Schlichtkrull et al. [2023], and SciFact Wadden et al. [2020] are used for evaluation only; no portion of these datasets is used for training or hyperparameter selection for the GNN or any other learned component.

References

Jon Barwise and John Perry. *Situations and Attitudes*. MIT Press, 1983.

Cristian Bodnar, Francesco Di Giovanni, Benjamin Paul Chamberlain, Pietro Liò, and Michael M. Bronstein. Neural sheaf diffusion: A topological perspective on heterophily and oversmoothing in GNNs. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, 2022.

Arthur P. Dempster. Upper and lower probabilities induced by a multivalued mapping. *The Annals of Mathematical Statistics*, 38(2):325–339, 1967. doi: 10.1214/aoms/1177698950.

Thibault Formal, Benjamin Piwowarski, and Stéphane Clinchant. SPLADE: Sparse lexical and expansion model for first stage ranking. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*, pages 2288–2292, 2021. doi: 10.1145/3404835.3463098.

Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, volume 70, pages 1321–1330, 2017.

Yunfan Guo et al. GraphCheck: Graph-based hallucination detection for large language models. *arXiv preprint arXiv:2407.16557*, 2024.

Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia d'Amato, Gerard de Melo, Claudio Gutierrez, Sabrina Kirrane, José Emilio Labra Gayo, Roberto Navigli, Sebastian Neumaier, Axel-Cyrille Ngonga Ngomo, Axel Polleres, Sabbir M. Rashid, Anisa Rula, Lukas Schmelzeisen, Juan Sequeda, Steffen Staab, and Antoine Zimmermann. Knowledge graphs. *ACM Computing Surveys*, 54(4): 71:1–71:37, 2021. doi: 10.1145/3447772.

Wanjun Hu et al. Adversarial fact-checking for evidence-aware fact verification. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023.

Yichen Jiang, Shikha Bordia, Zheng Zhong, Charles Dognin, Maneesh Singh, and Mohit Bansal. HoVer: A dataset for many-hop fact extraction and claim verification. In *Findings of the Association for Computational Linguistics: EMNLP*, pages 3441–3450, 2020. doi: 10.18653/v1/2020.findings-emnlp.309.

Ori Joren et al. Sufficient context: A new lens on retrieval augmented generation systems. In *Proceedings of the 13th International Conference on Learning Representations (ICLR)*, 2025.

Namgyu Ko, Jinheon Ye, Seanie Hong, Alexander J. Ratner, and Sung Ju Yoon. SelectLLM: Can LLMs select important instructions to annotate? In *Proceedings of the 13th International Conference on Learning Representations (ICLR)*, 2025.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9459–9474, 2020.

Zhenghao Liu, Chenyan Xiong, Maosong Sun, and Zhiyuan Liu. Fine-grained fact verification with kernel graph attention network. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 7342–7351, 2020. doi: 10.18653/v1/2020.acl-main.655.

Mahdi Pakdaman Naeini, Gregory Cooper, and Milos Hauskrecht. Obtaining well calibrated probabilities using Bayesian binning. In *Proceedings of the Twenty-Ninth AAAI Conference on Artificial Intelligence (AAAI)*, pages 2901–2907, 2015.

Liangming Pan, Yuyang Wu, Xinyuan Ma, Preslav Nakov, and Min-Yen Kan. Structured retrieval for iterative verification in evidence-based fact-checking. In *Findings of the Association for Computational Linguistics: EMNLP*, 2023.

Michael Schlichtkrull, Thomas N. Kipf, Peter Bloem, Rianne van den Berg, Ivan Titov, and Max Welling. Modeling relational data with graph convolutional networks. In *Proceedings of the 15th International Conference on the Semantic Web (ESWC)*, pages 593–607, 2018. doi: 10.1007/978-3-319-93417-4_38.

Michael Schlichtkrull, Zhijiang Guo, and Andreas Vlachos. AVeriTeC: A dataset for real-world claim verification with evidence from the web. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Michael Schlichtkrull, Zhijiang Guo, Andreas Vlachos, et al. AVeriTeC shared task 2024. Workshop on Fact Extraction and VERification (FEVER), EMNLP, 2024.

Murat Sensoy, Lance Kaplan, and Melih Kandemir. Evidential deep learning to quantify classification uncertainty. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 31, 2018.

Glenn Shafer. *A Mathematical Theory of Evidence*. Princeton University Press, 1976.

James Thorne, Andreas Vlachos, Christos Christodoulopoulos, and Arpit Mittal. FEVER: A large-scale dataset for fact extraction and VERification. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pages 809–819, 2018. doi: 10.18653/v1/N18-1074.

David Wadden, Shanchuan Lin, Kyle Lo, Lucy Lu Wang, Madeleine van Zuylen, Arman Cohan, and Hannaneh Hajishirzi. Fact or fiction: Verifying scientific claims. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 7534–7550, 2020. doi: 10.18653/v1/2020.emnlp-main.609.

Yichong Zheng, Xiaohu Fang, Shen Chen, Peifeng Jiang, and Xiaojun Wan. DREAM: Improving situational judgment in machine reading comprehension with scenario-based reasoning. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 2927–2942, 2020. doi: 10.18653/v1/2020.emnlp-main.237.

Junrui Zhou, Bangzheng Huang, Feng Li, Meishan Zhang, and Min Zhang. E-NER: Evidential deep learning for trustworthy named entity recognition. In *Findings of the Association for Computational Linguistics: ACL*, pages 2622–2634, 2023. doi: 10.18653/v1/2023.findings-acl.166.

Zhiwei Zhou, Zhijiang Guo, Mi Cheng, Yue Zhang, Weiyi Luo, and Bo Li. GEAR: Graph-based evidence aggregating and reasoning for fact verification. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 892–901, 2019. doi: 10.18653/v1/P19-1085.